2027年热门会议推荐

具体咨询：白老师 13516196409

智驱产品 技术领航

AI 时代产品和软件开发前沿系列峰会

核心定位

锚定AI 技术革新前沿，聚焦大模型与智能体工程化落地，解构 AI 时代软件工程架构革新路径，打通 AI 产品从技术突破到产品商业化验证的全链路。

关键词

- 大模型驱动
- 技术趋势
- Agent工程化
- 架构革新
- AI产品0-1
- 商业化验证

PM×AI 产品力领航者大会

2027年4月23-24·杭州

详情查看官网
演讲申报｜参会购票

Agentic AICon 智能体应用与架构工程大会

2027年6月18-19·上海

详情查看官网
演讲申报｜参会购票

Agentic AICon 智能体应用与架构工程大会

2027年11月5-6日·深圳

4月 5月 6月 9月 10月 11月

QECon 全球软件质量&效能大会

2027年5月21-22·深圳

QECon 全球软件质量&效能大会

2027年9月10-11日·上海

QECon 全球软件质量&效能大会

2027年10月15-16日·北京

演讲申报｜参会购票
详情查看官网

AI 赋能 提质增效

QECon全球软件质量&效能大会

核心定位

聚焦 AI 技术在软件研发全流程的落地应用，以“质量保障&效能提升”为双驱动，助力研发团队实现软件研发的智能化转型。

关键词

- AI 赋能研发
- 效能提升
- 测试驱动
- 平台工具链
- 质量保障
- .....

生产级Agent设计蓝皮书·行业案例卷

"蓝皮书共建计划"启动——行业与场景专家 招募中

白皮书定义模式，蓝皮书记录落地。
把你的生产案例，写进行业参照系。
你的案例，很可能是这个行业第一份模式应用实践指南。

交流地点：《艾普·AI生产力研究院》展位咨询

扫码参与



共建计划

以真实案例为基础，持续组织行业与场景的 Agent 设计模式应用撰写。蓝皮书不写概念，只记录具名系统——它面对什么约束、踩过什么失败、为什么这样选架构、哪些经验可以迁移到你的行业。

我们招募这样的行业与场景专家

主导或深度参与过 Agent 系统建设，有真实跑在生产环境的案例；
在强监管、高可靠或大规模部署约束下做过工程取舍（金融、制造、政务、医疗、能源、电商等皆宜）；
愿意把一段经验，沉淀成可评审、可迁移的模式案例。

参与共建，你将获得

署名蓝皮书案例贡献者，成果进入 ADPS 公共技术资料长期存档；
用白皮书的模式语言系统梳理自己的实践，让团队经验成为行业参照；
参与共同体模块研讨与模式评审，共同界定下一版模式的边界；
与已在共同体的腾讯、蚂蚁、百度、京东等一线专家跨行业互审。

《企业级 Agent 设计白皮书》发布

纸质版 & 电子版 同步上线

领取地点:《艾普·AI生产力研究院》展位



Agent 正在从「会回答问题的模型」变成「能参与业务流程的系统」。模型仍是引擎，但决定它在生产中稳不稳的，是周围这套工程结构。本白皮书给出智能体工程的可复用架构词汇与评审标准。

27	7×6	3	5	6
矩阵模式规范	认知功能 × 执行拓扑	横切工程面 X1–X3	工程契约	步选型法

智能体工程的「GoF设计模式」

27个可复用Agent设计模式双轴框架

企业级智能体架构评审与选型实战手册

双轴框架

纵轴 · 认知功能

Agent 在做什么类型的工作

感知 Perception　记忆 Memory　推理 Reasoning　行动 Action

反思 Reflection　协作 Collaboration　治理 Governance

横轴 · 执行拓扑

数据与控制怎么流动

链式 · 顺序　路由 · 分流　并行 · 并发　编排 · 协调

循环 · 迭代　层级 · 分层

获取白皮书电子版

艾普·AI生产力研究院
Artificial Intelligence Institute of Productivity

Artificial Intelligence Institute of Productivity · 中立 · 开放 · 共创

面向 AI 原生软件工程的开放研究与实践组织

驾驭智能·定义 AI 时代新质生产力

深度承接《"人工智能+软件"专项行动实施方案》的专业化研究与交流平台

软件质量与效能 | **智能体工程与应用** | **六大研究方向** | **五大核心模块**

艾普 · AI 生产力研究院（AIIP）是一个中立、开放、共创的专业化研究与交流平台。研究院依托在软件质量与效能、智能体工程与应用等前沿领域的长期实践积累与前瞻研究底蕴，汇聚资深技术专家与企业研发团队等各方力量，共同推进行业能力体系建设、参考标准研究与评估框架研制，致力于成为支撑智能软件产业新质生产力发展的专业化交流平台。

更多详情请查看官网：https://aiip.org.cn/

关注官网

申请加入

与先行者同行，共赴 AI 生产力浪潮

六大研究方向

AI for SE
智能软件工程

AI赋能软件开发全流程、AI Coding 生产力与质量保障、智能测试工程，对标智能编程工具链与智能化技改。

度量体系
质效度量与治理

AI 质效度量模型、Token 成本治理、ROI 分析框架，以及成熟度评估参考框架的研究与共建。

ADPS
Agent 设计模式

Agentic 设计模式、多智能体架构、AI 系统技术栈与最佳实践沉淀，对标技能包与应用商店。

FDE
前线部署工程师

工程实践方法论、行业场景适配、企业落地案例与效能提升路径，对标就业友好与重点行业落地。

SE for AI
AI 工程化落地

AI 系统全生命周期工程化方法，可靠性与质量保障，以及 MLOps / LLMOps 交付与治理框架。

AI for X
AI 生产力

AI深度赋能千行百业，从软件研发走向办公、运营、数据、科研等真实业务价值场景，把工程能力转化为可度量的生产力。

AI 质效宣言

发起单位：QECon组委会 QECon Organizing Committee

从共识到行动，让宣言融入每一次研发实践

回归研发本质 **重塑人机协作** **推动行业共识**

协作共生 高于 局部取代
长效质量 高于 短期提效
适应进化 高于 固化流程
价值创造 高于 技术崇拜
数据驱动 高于 经验依赖
开放共享 高于 封闭自建

加入共建计划
获取电子画面

两层能力递进 · 一层工程底座

AI 增强软件研发全景架构

AI 嵌入 SDLC · 效能提升

• 人主导 • AI 协同

需求
- 行业调研与趋势分析
- 创意启发与功能探索
- 需求分析与规格细化
- 需求检查与内容评审

设计
- 代码理解与逻辑探测
- 技术选型与架构设计
- 接口设计与数据模型生成
- 技术方案评审与风险分析

开发
- 代码生成与缺陷解决
- 氛围编程与规约编程
- 单元测试生成与执行
- 代码评审与问题修复

测试
- 测试用例与数据生成
- 测试脚本修复与自愈
- 端到端测试生成与维护
- 缺陷发现与根因分析

CI / CD
- 构建与部署脚本生成
- 流水线失败诊断与修复
- 无人值守部署与发版
- 依赖升级与漏洞修复

运维 / 运营
- 告警聚合与摘要生成
- 故障分析与定位
- 故障处理与系统自愈
- 用户体验与反馈洞察

AI 研发范式变革 • 效能跃迁

• AI 主导 • 人监督

工程实践
- 驾驭工程 （模型决定上限，Harness 提升底线）
- 知识工程 （跨仓库代码与文档索引 · 沉淀复用）
- 验证闭环 （生成 → 验证 → 修正）
- 质量门禁 （构建多层次质量防护网）

流程重构
- 意图驱动、面向结果的交付 （替代多阶段接力）
- 对齐在前、执行在后 （人谈清边界，AI 自主实现）
- 需求即规格、证据即信任 （规格成为人机契约）
- 快速原型、持续迭代 （沿用敏捷 · 精益研发思想）

组织重塑
- 交付层抹平前后端分工 （跨栈交付 · 功能负责人）
- 架构师专注标准与门禁 （定标准 · 守门禁）
- 设计师与产品自主交付 （部分需求不经研发）
- 先锋队试点后规模复制 （先跑通再逐步推广）

效能度量
- 结果面：多 · 快 · 好 · 省
- 过程面：渗透率 · 成熟度 · 使用效果
- 数据源：代码 · 会话 · 配置 · 需求 · 监控
- 北极星：需求交付周期 · 单位 token 有效价值

AI 工程底座 • SE4AI （Software Engineering for AI）

• 自下而上 • 决定能力上限

模型层
- 多模型接入与路由 （模型网关 · 按能力与成本择优）
- 能力分级与降级 （强弱模型分级 · 故障回退）
- 领域微调与私有化 （微调训练 · 内网部署）
- 模型评测与选型 （基准跑分 · 场景实测对比）

Agent 层
- Agent 架构与编排 （单/多 Agent · 规划与控制流）
- 上下文工程与记忆 （上下文组装压缩 · 长短记忆）
- 运行时沙盒与工具接入 （隔离执行 · MCP 与技能调用）
- 灰度发布与回滚 （能力评估达标才放量 · 版本化 · 自动回退）

生态层
- 技能与智能体市场 （技能复用 · Agent 共享流通）
- MCP Hub 与工具生态 （工具目录 · 权限与调用治理）
- 知识资产与语料沉淀 （业务知识 · 代码知识 · 过程知识）
- 检索服务与知识图谱 （语义检索 · 图谱关系推理）

贯穿三层 | **安全与治理** 数据脱敏 · 提示注入防护 · 最小授权 · 输出合规 | **可观测性** 决策链路追踪 · 上下文溯源 · 失败归因 | **能力评估** 任务成功率 · 人工接管率 · 单任务成本

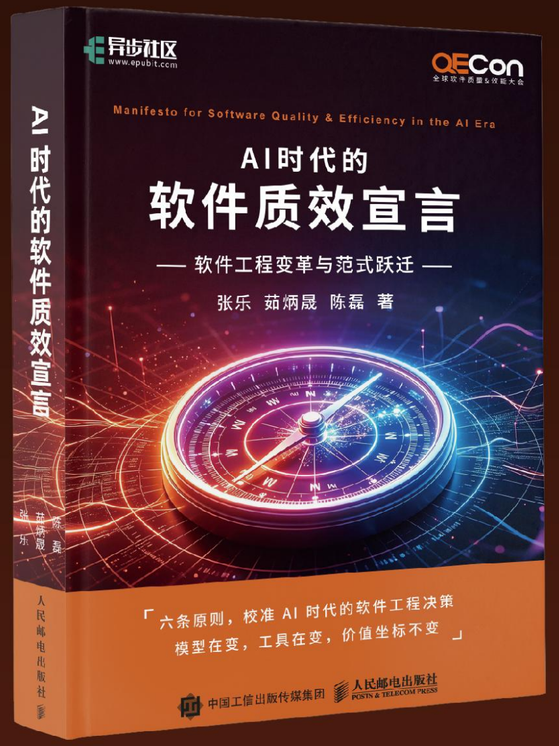
为每一位实践者提供一组稳定的价值坐标——当工具在变、模型在变、最佳实践在变的时候，这些原则帮你判断什么是不变的、什么是值得坚守的。

AI 时代的软件质效宣言

新书发布

— 软件工程变革与范式跃进 —

扫码订阅



系列电子书结构·三篇十九章 (道 · 术 · 势 —— 一条完整的 AI 研发变革路径)

道

第一篇 · 解读篇 (1-8 章)

确立价值观与原则：理解 AI 时代软件质效的本质，重建研发协作的信念底座，是全书的「道」。

术

即将发布

第二篇 · 单点提效篇 (9-14 章)

覆盖需求分析 → 架构设计 → 编码实现 → 测试验证 → CI/CD → 运维运营的关键环节，打通 AI 赋能研发全链路，是全书的「术」。

势

即将发布

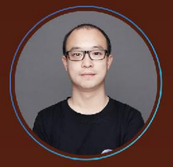
第三篇 · 范式篇 (15-19 章)

当 AI 贯穿整条研发链路之后，组织形态本身将如何变革？从团队结构到协作范式，预见未来，是全书的「势」。

—— 主编 ——



张乐
腾讯
智能化软件工程
资深技术专家



茹炳晟
腾讯研究院
高级技术顾问



陈磊
前京东测试架构师

—— 作者 ——



王玉霞
北京兴云数科
过程改进专家



李佳奇
去哪儿旅行
技术总监



牛万鹏
百度
Comate研发经理



张晔
腾讯端服务质效
技术负责人



张涛
快手质量通道委员



付求爱
华为云
AIOps技术负责人



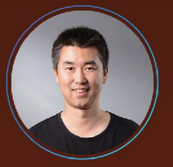
李萌
珊瑚智成
创始人&CEO



张刚
上海英慕科技
资深技术专家



张兆鑫
OPPO AI架构师



王磊
华为 AI技术专家

主办方：智盟创课

智盟创课（北京）科技有限公司（简称“智盟 Stu.”），作为专业IT产研培训服务供应商与AI时代数字化人才发展伙伴，为科技研发型企业及各类组织提供系统性、分层次、重实战的人才培养解决方案，同时搭建行业前沿技术交流平台，致力于成为企业数字化转型进程中人才培养与业务创新的核心支撑力量，推动组织实现成功转型及经营业绩持续增长。

同时，智盟发起并组织三大 IT 领域品牌会议：QECon 软件质量效能大会、SECon+AgentX 智能时代软件工程技术大会、PMxAI 产品力领航者大会、Agentic AICon 智能体应用与架构工程大会，助力产研中心深化数字化转型、聚焦业务价值达成，在产品研发的快速交付与高质量发展之间找到平衡，打通企业全维度学习视角，最终实现人才能力提升与企业竞争优势增强的双重目标。



详情咨询: 010-65798049

多元化学习形式

技术峰会 → 公开课 → 内训定制 → 微咨询 → 在线课程

核心培训方向

AI大模型应用 → 产品运营提升 → 数据分析决策 → 通用管理赋能 → AI职场综合应用

软件研发赋能

大模型前沿	架构设计	数据库	产品经理	持续运维	质量管理
移动开发	组织管理	云计算	开发编程	敏捷开发	大数据

AGENTIC AICON · 深圳站 · 2026.10

可信交付

AI Coding 时代交付质量的实践与思考

许欣雅 · 蚂蚁集团芝麻信用 · 测试开发专家

关于我

长期负责支付宝营销平台、芝麻信用等大型业务的质量保障

当前专注 AI Coding 质量体系建设

这场分享，你能带走什么

01 一份风险清单 —— 生成通过了，为什么还不能交付?

02 一套监督设计 —— Agent 越自主，交付如何不失控?

03 一份实践思考 —— 今天如何交得更好，明天交付会变成什么样?

01 · 挑战 · 业务场景演绎

一次增量支付订单的佣金调整

佣金怎么算、页面怎么展示、账单怎么出，都澄清了。

Agent 完成修改，新订单验证通过。

但老订单的结算金额，也跟着变了。

“新订单采用最新规则”，被自行扩成了“所有订单都适用”。

需求没说清的边界，
Agent 自主定义了边界

01 · 挑战 · 可信

AI 参与生产，交付新增哪些风险?

AI 引入或放大的风险 · 标签为常见关联机制

01 **越界修改** 过度推断 案例 1
需求没说要改，Agent 却改了原有行为。

02 **验证漏项** 同源盲区 案例 1
新功能测过了，受影响的老功能没测。

03 **凭空编造** 幻觉
接口不存在、工具没执行，却给出了完成结论。

04 **前后矛盾** 上下文漂移
需求说一套，设计、代码和测试各做一套。

05 **破坏架构** 局部优化
为了完成局部功能，重复造轮子、绕过分层。

06 **安全漏洞** 安全知识与推理不足
功能跑通了，权限校验、敏感数据保护却漏了。

局部验证通过，不代表整体结果可信。

行业证据 · 分别观察约束、安全与验证

65.4% 未稳定遵循规范
受访开发者反馈：Agent 未始终遵循组织规范。
Qodo · 2026.09 · 500 位美国开发者

56% 安全测试通过率
AI 生成代码在安全测试中的平均通过率。
Veracode · 2026 · 代码生成安全测试

74.3% 测试漏检回滚
受访工程负责人曾因此回滚 AI 生成代码。
Checksum · 2026.07 · 105 位工程负责人

01 · 挑战 · 业务场景演绎

借助 Agent，业务自己就能做出 Demo / 产品

想让真实用户试一试，快速验证想法的业务价值。

但交给真实用户之前，风险有多大，还说不清。

异常场景会不会出错?
接入原有业务流程，会不会冲突?
换一批真实用户，稳定性和体验还能保证吗?

想法已经做成产品，
怎样才能放心交给用户验证?

01 · 挑战 · 交付

产品生成之后，交付还有哪些未知风险?

验证范围从一个功能，扩到完整业务流程，再进入真实用户与流量。

01	功能正确性 一个功能	主流程能跑，其他情况呢?	异常分支、并发与重复请求 存量行为和数据是否仍然正确
02	完整产品可用性 整条业务流程	单个功能可用，串起来呢?	支付、退款、结算等上下游衔接 状态、金额与业务规则是否一致
03	真实流量表现 真实用户与运行环境	演示用户通过，换成人群与流量呢?	不同人群、真实数据、高峰流量 稳定性、响应体验与运行成本是否可接受

共同交付条件	权限与数据安全 · 监控告警 · 灰度与回退
上线后持续验证	用户体验、业务转化与成本收益

从可以演示，到真实可用，每一层未知都需要验证依据。

01 · 挑战 · 可信交付

可信交付是业务AGI的基本要求

让 Agent 在明确边界内自主执行，将生成成果交付为真实可用的业务服务。

可信

自主执行有边界，执行结果有证据

业务约束贯穿理解、生成与执行。
结果独立核验，不能只凭 Agent 的完成声明。

交付

完整业务能运行，业务价值能验证

接通完整业务流程，核实存量兼容与真实运行表现。
让真实用户用起来，验证想法的业务价值。

业务 AGI 链路的三个构建要求

01

全过程监督

让自主生产不偏离目标

沿生成与执行过程取证核验
关键动作及时介入
最终结果对照业务目标验收

02

持续演进

让现有链路持续有效

运行反馈发现瓶颈与失效机制
通过评测吸纳新模型与新策略
受控更新，无效机制及时退出

03

可集成生态

一个场景跑通，其他业务快速复用

复用统一底座与通用审查基线
按需接入知识、规则、Skill 与工具
快速扩展到不同业务场景

以业务场景验证有效性，通过持续演进，快速适应业务与环境（Harness 和 LLM）的发展。

Agentic Coding Supervisor 交付监督者

面向自主交付的监督者，贯穿生成、执行与反馈改进。

生产侧 · Harness / Workflow

需求 → 设计 → 编码 → 测试 → 修复 → 验收 → 发布 → 运行

01 全过程监督

运行事件与执行证据

运行时观测 模型 / Skill / 知识召回 / 上下文
工具调用 / 压缩 / 重试 / 耗时

AxLint · 独立审查引擎

独立上下文：原始目标、业务约束、变更快照与真实执行凭据

边界异常　真实性　一致性　架构　安全可靠　不确定性

静态检查
资金平衡 / 接口契约
架构约束 / 重复实现

动态验证
工具是否实际调用
异常路径与存量回归

智能评测
Spec 遵从 / 对抗 / 轨迹
LLM-as-Judge

AxLint 规则 全局基线 + 项目规则，按阶段与风险加载

结论 + 证据 + 处置 继续 / 补验 / 限定修正 / 人工接管

处置回传

02 持续演进

度量与反馈校准

输入 需求、Spec 与业务约束
过程 模型、Skill 与上下文
结果 测试凭据与真实结果

Reflexion

- **收集反馈**
  Badcase / 误报漏报
  人工改判 / 运行耗时
- **评测与回放**
  规则、模型、验证器
  对照基线验证改进
- **受控更新**
  审核 / 版本 / 灰度 / 回退

← 验证后更新规则与运行策略

03 可集成生态

统一接口 · 版本管理

知识与上下文	规则与评测资产	Skill 与工具	可插拔 Provider
业务知识 / Spec / Memory	规则 / 数据集 / Badcase	业务能力 / 验证工具	模型 / 验证器 / 执行环境

接入过程，看见事实；独立核验，返回处置；反馈验证后，再更新能力。

独立的 Runtime，伴随 Agent 执行

上下文隔离，事件与证据相连。

生成与执行

生产 Agent

执行上下文

任务规划与生成
修改与自我修正
任务完成报告

计划 → 执行 → 验证

上下文隔离
独立判断

动态 Hook
事件 + 执行证据

⟶
关键动作前后触发

处置返回任务

⟵
补验 / 修正 / 接管

独立监督

ACS Runtime

审查上下文
原始目标与业务约束
变更快照、适用规则与真实执行凭据

调用 AxLint
关键风险核验后放行

按事件与风险触发
补验 / 修正 / 接管

运行时观测
采集过程数据，
支持异常核验与 Reflexion 分析

工具调用
上下文占用
知识召回
目标与约束偏离

独立上下文减少共同盲区；关键风险及时核验，运行信号持续采集。

AxLint：沿交付链路组织，按业务风险装配

以已确认的目标与约束为基线，区分需求变更与生成漂移，核对实现与验证是否同步。

需求评审	研发分析	代码实现	质量验证	发布运维
ADocLint 文档与设计		**ACodeLint**	**ATestLint**	**AServiceLint**
开篇订单示例 · 设计中的漂移 **边界被扩大** 适用范围尚未明确， 设计却按"所有订单适用"展开。 核对需求与设计的适用边界		**影响被扩散** 公共逻辑被修改， 老订单也受影响。 核对实际改动 与约定范围	**验证未覆盖** 只验证新订单， 未验证存量影响。 核对验证范围 与实际改动	**发布检查** 配置 / 容量 / 回退 → 安全与可靠性核对 **告警与异动监测** 业务 / 运行指标 → 基线与异动核对

专项 AxLint	资金与数据	高可用	安全	体验	模型
跨阶段按需加载	资金平衡 · 数据一致性	幂等 · 容量 · 应急	越权 · 注入 · 泄露	交互链路 · 响应体验	幻觉 · 上下文 · 越界

静态检查
核对架构、规范与安全约束

动态验证
核实真实执行、边界与存量影响

智能评测
评估语义与轨迹，保留不确定性

边界扩大 → 影响扩散 → 验证未覆盖：逐步自洽，不等于始终符合原目标。

AxLint：规则组成、生成与实例维护

人工沉淀、自动挖掘与运行反馈，形成可按任务调用的审查资产。

生成机制

多种来源形成候选，
验证后进入资产池。

人工沉淀

企业标准、专家经验
业务知识、手工维护

自动挖掘

故障、漏洞、代码模式

> **典型：历史 commit 提炼 bugfix 模式**

修复前后 + 反例验证

运行反馈

问题标注、人工改判
误报漏报、覆盖缺口

AxLint · Rule + Tool

静态规则 + 动态 LLM 规则

规则定义约束；工具调用模型、Skill 与 Agent 完成核验。

公司编码规范	专家经验	线上故障经验
资金风险	技术栈经验	应用代码挖掘
反馈改进产物	漏洞情报	.....

审查资产池

保留来源、版本、适用阶段、命中结果、
人工改判与改进提案。

全局基线
复用共性标准

业务域规则
沉淀领域约束

项目规则
补充项目约束

按任务、阶段与风险加载，限定适用范围。

实例维护

每个实例可追溯，
维护有效性与适用范围。

定义适用范围

项目 / 阶段 / 触发条件
明确维护责任人

保留使用记录

命中证据、误报反馈
人工改判与覆盖缺口

维护版本与样本

规则与工具版本对应
保留修复前后及反例

失效与退出

业务变更后重新验证
过期停用，版本可回退

把经验沉淀成可组织、可调用、可评估、可更新的审查资产。

Reflexion：审视生产与监督两侧 Runtime

既检查生产侧如何生成与执行，也检查 ACS 自身如何审查与干预。

生产侧 Runtime / Harness

约束遵循、上下文、工具调用
生成偏差、无效重试与执行成本

ACS 自身 Runtime / Harness

风险检出、误报漏报、处置有效性
触发策略、审查耗时与阻塞成本

↓ 两侧运行反馈，进入同一条受控改进流程 ↓

归因

区分生产偏差
与监督机制问题

→

提出改进

改进上下文、规则
工具与运行策略

→

回放评测

同一任务集对照基线
比较约束遵循、漏报与耗时

→

受控更新

审核 / 版本 / 灰度
不达标则回退

↰ 持续观察新版本效果，反馈进入下一轮；无效经验降级或退出

两条回路分开

任务内：保留目标、限定修正、独立复验
跨任务：验证改进、更新能力；自主权限不随优化自动扩大

两侧都可提出改进；效果经评测、变更经授权，再更新对应 Runtime。

集成生态：统一 Runtime，按业务装配能力

业务明确目标、边界与验收标准，通过插件扩展监督能力，保留已有研发工具链。

Hook 策略
哪些事件触发审查
何时等待、提示或拦截

观测重点
业务关心哪些风险信号
采集点、阈值与证据来源

规则与评测
全局基线 + 项目约束
正反例、验证器与评测集

知识与 Memory
业务知识、Spec、历史经验
来源、版本、适用条件与有效期

Skill
业务分析与验证方法
按任务与权限调用

工具与执行环境
业务查询、测试执行、日志取证
明确调用权限、执行环境与返回证据

↓ 按项目、任务与阶段装配 ↓

ACS Runtime · 可集成的交付监督底座

模型与验证能力可替换

Runtime
Hook 策略与观测重点

AxLint
知识、规则、Skill 与工具

Reflexion
运行反馈与评测资产

进入真实业务任务

选定场景 → 装配能力 → 执行与审查 → 复验结果 → 反馈校准
接入效果看需求覆盖、风险识别与处置成本，再逐步扩大范围。

统一任务、证据与权限契约，让业务能力进入监督、核验与反馈闭环。

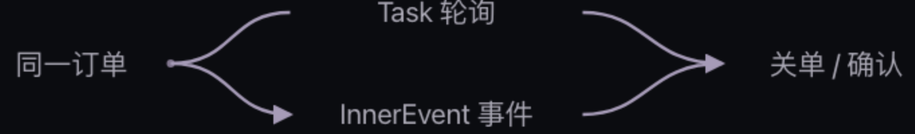
03 · 落地效果与经验分享

真实拦截：让业务约束真正进入实现

从两个真实拦截，看约束是否落实、系统实现设计是否前后一致。

业务约束未落实

两条通道处理同一订单，缺少防重复机制



需求	设计	生码
冲突策略	幂等保护	调用前
未定义	未设计	未做去重

跨阶段核验约束落实

核对需求、设计与代码，识别防重复保护缺口。

系统实现设计前后不一致

同一系统实现设计写明按类目灰度，开关设计却仅能全局控制

系统实现设计 · 灰度策略		系统实现设计 · 开关设计
只向指定类目开放	≠	全局功能开关 全局事件开关

策略声明	开关设计	过滤设计
按类目	仅有	类目过滤
小范围开放	全局开关	未设计

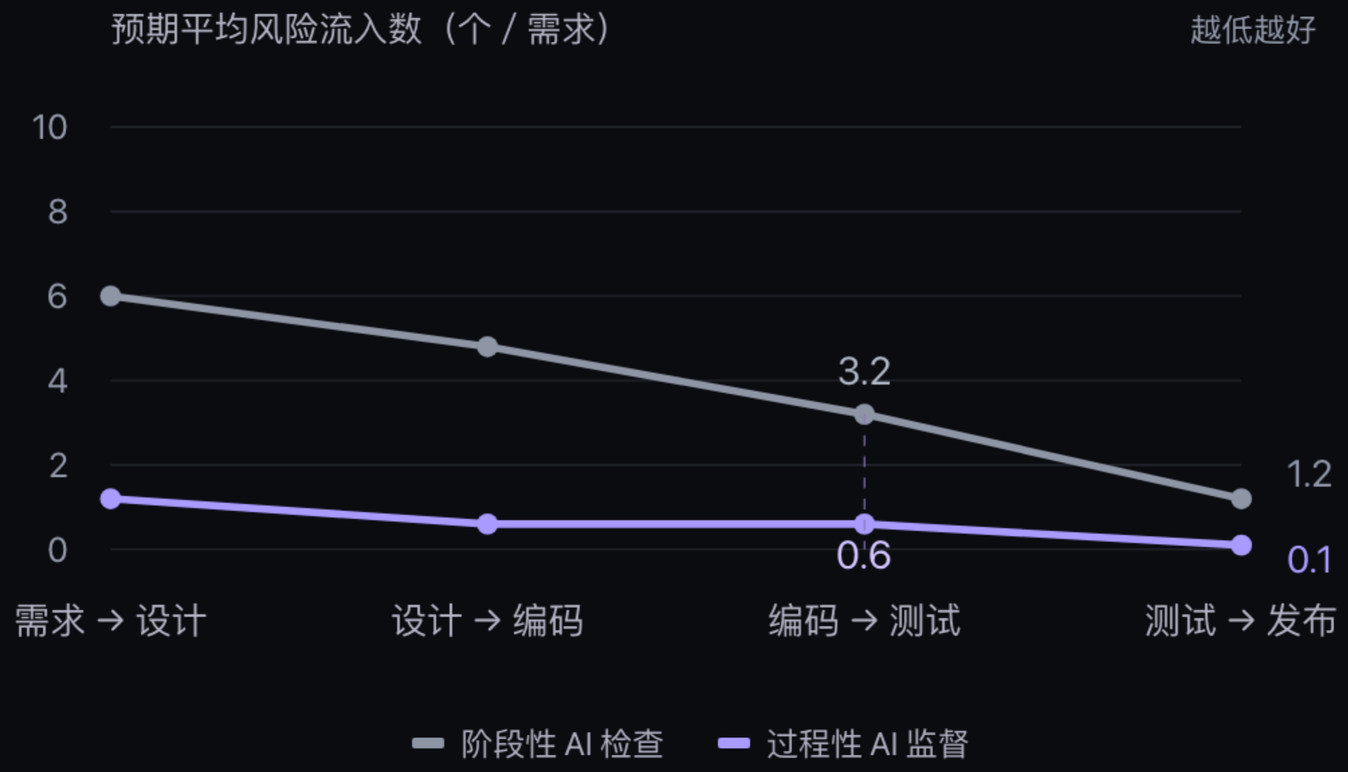
核验同一系统实现设计内的一致性

对照灰度策略与开关、过滤设计，识别前后矛盾。

在当前阶段识别并处置偏差，减少问题向后传递。

03 · 落地效果与经验分享

过程监督，让风险少穿透一层



编码 → 测试
平均流入风险（个 / 需求）

3.2 → **0.6**

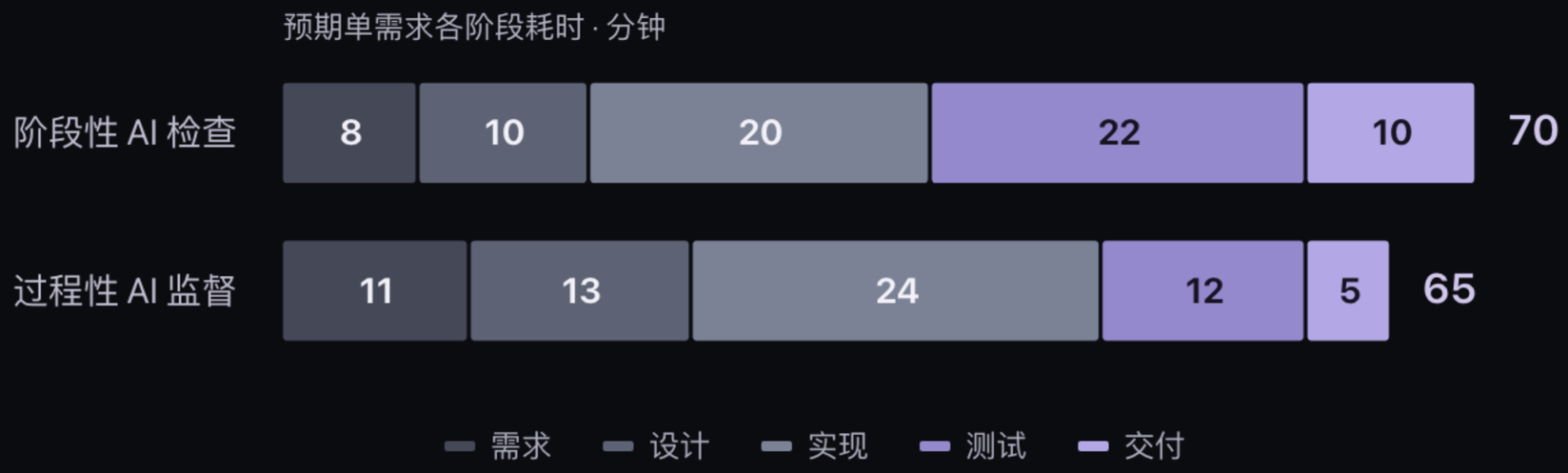
生成与执行中及时核验，
交接前完成风险处置。

发现只是起点，减少风险向后传递才是目的。

03 · 落地效果与经验分享

交付效率：把时间花在前面，减少后段反复

沿需求、设计、实现、测试与交付，比较两种 AI 监督方式的时间分布。



需求

先确认业务边界

增加范围与验收澄清
减少带着歧义往下做

设计

提前核对约束

增加依赖与异常审查
减少错误方案的延伸

实现

边生成，边纠偏

增加过程核验与调整
减少错误方向的尝试

测试

少回改，仍要验

减少缺陷定位与回改
保留必要测试与复验

交付

少补救，守准出

减少遗留问题的补救
保留准出与发布验证

前段多做确认与核验，后段少做纠错与补救。

03 · 落地效果与经验分享 · Reflexion

Reflexion：规则与 Harness 的自我改进

一次误拦，一次漏检：分别改进规则判断与上下文恢复。

规则把提醒判成了阻塞

后台自动关单任务，被外部依赖保护规则拦住

内部服务
查询订单状态 → 后台异步任务
自动关闭订单

人工反馈
内部服务
后台异步

Reflexion 归因
风险级别
判得过重

规则更新 · 已应用
阻塞
↓ 提醒

把一次误报反馈，写回项目规则

已更新严重级别；保留风险提示，调整阻塞要求。

规则还在，检查却漏了

案例推演 · 压缩后漏做脱敏检查，放过了完整手机号

新增代码
日志含完整手机号 → 脱敏检查遗漏
审查通过

Reflexion 排查
规则文件还在
检查入口丢了

ACS 补齐恢复
压缩后
补回 Skill 清单

Engine 新版承接
原生恢复清单
回放后撤补丁

Claude Code 2.1.304 已支持清单恢复

回放确认后，由原生恢复替代额外补丁。

把一次反馈，变成下一次执行的改进。

03 · 落地效果与经验分享 · 演进设计

让交付链路持续受益于 LLM 与 Harness 的进步

业务积累持续复用，具体实现通过评测更新。

吸纳原生进步

LLM 与 Harness 能力升级

指令遵循、上下文容量等
兼容接入或轻量适配

→

回放验证效果

同一组业务样本

比较核验效果与执行成本

→

采用更优实现

验证通过后，受控更新

按授权采用，效果退化可回退

同步退出：原生能力已覆盖的重复补丁

↑

持续复用的业务积累　　业务约束　　领域知识　　评测样本

原生恢复 Skill 清单经回放验证后，可替代 ACS 的额外恢复补丁。

业务积累留下来，**交付能力持续升级。**

03 · 落地效果与经验分享 · 演进思考

交付，从一个版本走向每次请求

生成从研发进入使用过程，监督范围也从代码扩展到业务行动与每次请求。

L3 交付产品

当前实践 · 人验证

“增加定向发券功能。”

功能先做好，用户再使用

代码先生成，
经过验证后发布

交付什么

一个功能版本

L4 交付业务目标

演进设想 · 人监督

“预算内，提升指定人群复购。”

业务给目标，Agent 自主交付

生成方案后，
直接采取业务行动

交付什么

一次业务任务的结果

L5 交付用户个性化需求

开放推演 · 人治理

“按我的使用计划，组合权益。”

根据用户需求，即时交付

请求到达后，
局部业务逻辑才生成

交付什么

本次请求的服务结果

ACS Runtime · 监督随生成进入业务运行

生成中纠偏，发布前验收

核对需求、改动与验证证据

行动前核验，执行中限权

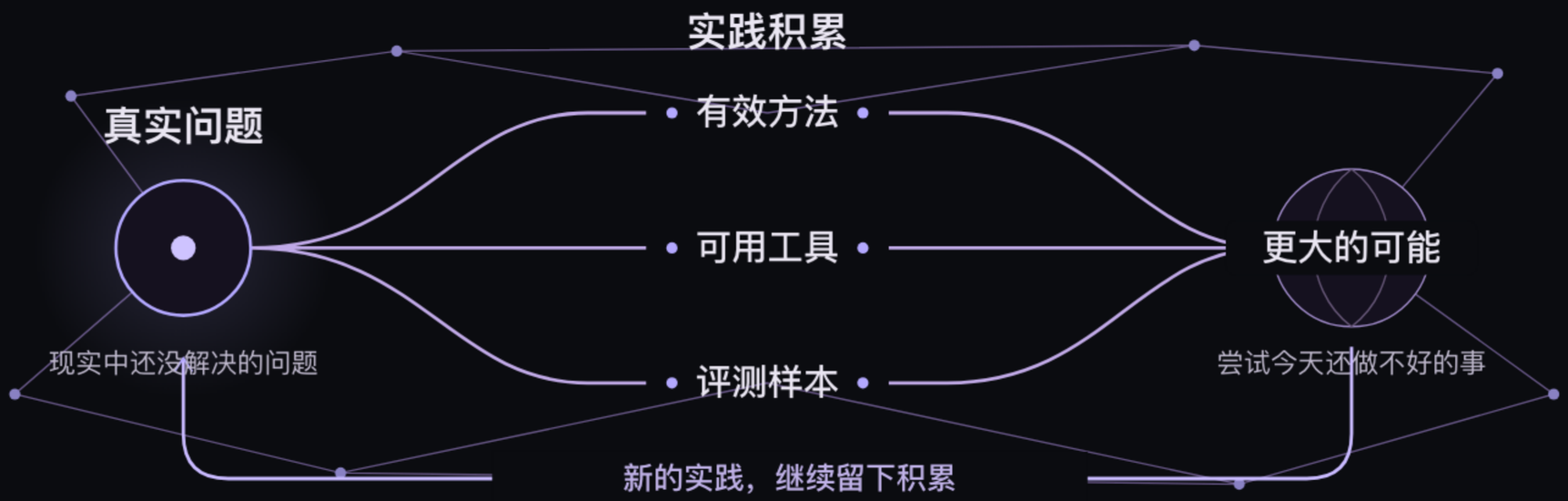
核对目标与预算，确认真实执行与结果

请求内核验，不确定则降级

独立审查、隔离执行，保留回退路径

03 · 实践之后，面向未来

模型进步这么快，我们该积累什么？



模型向前走，我们能解决的问题也跟着多一些。

参与 AGI 的建设，和不断进步的智能一起，**做成更多今天做不到的事。**

欢迎交流讨论

许欣雅 · 蚂蚁集团芝麻信用